

Лабораторна робота №5

Тема: Аутентифікація та авторизація. Робота з Web API. JSON Web Token.

Мета: набути навичок роботи з механізмами аутентифікації та авторизації користувачів в ASP.NET, набути навичок роботи з Web API.

Хід роботи:

Завдання 1.

Реалізувати реєстрацію, автентифікацію та авторизацію користувача з використанням пакету Microsoft Identity.

1: Встановлення пакету Identity

Спочатку встановимо пакет, як вказано у завданні.

Виконай у терміналі в папці проекту (SurveyPortal):

dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 8.0.

```
E:\University\4_course\Asp_net\lab2\SurveySln\SurveyPortal>dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 8.0.0
SurveyPortal succeeded with 1 warning(s) (0,2s)
C:\Program Files\dotnet\sdk\9.0.306\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.Sdk.FrameworkReferenceResolution.targets(82,5): warning NETSDK1086: A FrameworkReference for 'Microsoft.AspNetCore.App' was included in the project. This is implicitly referenced by the .NET SDK and you do not typically need to reference it from your project. For more information, see https://aka.ms/sdkimplicitrefs
Build succeeded with 1 warning(s) in 1,3s
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.AspNetCore.Identity.EntityFrameworkCore' into project 'E:\University\4_course\Asp_net\lab2\SurveySln\SurveyPortal\SurveyPortal.csproj'.
info : Restoring packages for E:\University\4_course\Asp_net\lab2\SurveySln\SurveyPortal\SurveyPortal.csproj...
```

2: Створення AppIdentityDbContext

Цей клас відповідатиме за таблиці Identity (користувачі, ролі тощо).

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace SurveyPortal.Models
{
    public class AppIdentityDbContext : IdentityDbContext<IdentityUser>
    {
        public AppIdentityDbContext(
            DbContextOptions<AppIdentityDbContext> options)
            : base(options) { }
    }
}
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.2				
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з лабораторної роботи	Літ.	Арк.	Аркушів	
Розроб.		Вещиков О.М.							
Перевір.		Українець М.О.					1	9	
Керівник						ФІКТ, гр. ІПЗ-22-2			
Н. контр.									
Затверд.									

3: Оновлення Program.cs (Конфігурація Identity)\

```
using Microsoft.EntityFrameworkCore;
using SurveyPortal.Models; // <-- ОСЬ ЦЕЙ РЯДОК ВИПРАВЛЯЄ ПОМИЛКУ
using Microsoft.AspNetCore.Identity;

var builder = WebApplication.CreateBuilder(args);

// 1. MVC, DbContext та Репозиторій (з минулих лаб)
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<SurveyDbContext>(opts => {
    opts.UseSqlServer(builder.Configuration["ConnectionStrings:SurveyPortalConnection"]);
});
builder.Services.AddScoped<ISurveyRepository, EFSurveyRepository>();

// 2. Сервіси Сесій (з минулих лаб)
builder.Services.AddSingleton<HttpContextAccessor, HttpContextAccessor>();
// Цей рядок (рядок 17) тепер буде працювати
builder.Services.AddScoped<SurveySession>(sp => SessionSurveySession.GetSession(sp));
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

// 3. Конфігурація Identity
builder.Services.AddDbContext<AppIdentityDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration["ConnectionStrings:IdentityConnection"]
    );

// Додаємо Identity з підтримкою РОЛЕЙ
builder.Services.AddIdentity<IdentityUser, IdentityRole>(opts =>
{
    opts.Password.RequiredLength = 8;
    opts.Password.RequireDigit = true;
    opts.Password.RequireUppercase = true;
    opts.Password.RequireNonAlphanumeric = false;
    opts.User.RequireUniqueEmail = true;
})
    .AddEntityFrameworkStores<AppIdentityDbContext>()
    .AddDefaultTokenProviders();

var app = builder.Build();

app.UseStaticFiles();
app.UseSession();

app.UseAuthentication();
app.UseAuthorization();

app.MapDefaultControllerRoute();

SeedData.EnsurePopulated(app);
// Створення ролей та Адміна
await IdentitySeedData.EnsurePopulatedAsync(app);

app.Run();
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
						2
Змн.	Арк.	№ докум.	Підпис	Дата		

Створення IdentitySeedData (Створення ролей та Admin)

using Microsoft.AspNetCore.Identity;

namespace SurveyPortal.Models

```
{
    public static class IdentitySeedData
    {
        // Імена ролей
        private const string adminRole = "Admin";
        private const string userRole = "User";

        // Дані нашого адміна
        private const string adminUser = "Admin";
        private const string adminEmail = "admin@survey.com";
        private const string adminPassword = "AdminPassword123";

        public static async Task EnsurePopulatedAsync(IApplicationBuilder app)
        {
            // Отримуємо сервіси, необхідні для роботи з Identity
            using (var scope = app.ApplicationServices.CreateScope())
            {
                var roleManager = scope.ServiceProvider
                    .GetRequiredService<RoleManager<IdentityRole>>();
                var userManager = scope.ServiceProvider
                    .GetRequiredService<UserManager<IdentityUser>>();

                // 1. Створення ролі "User"
                if (!await roleManager.RoleExistsAsync(userRole))
                {
                    await roleManager.CreateAsync(new IdentityRole(userRole));
                }

                // 2. Створення ролі "Admin"
                if (!await roleManager.RoleExistsAsync(adminRole))
                {
                    await roleManager.CreateAsync(new IdentityRole(adminRole));
                }

                // 3. Створення користувача-адміна
                IdentityUser? user = await userManager.FindByNameAsync(adminUser);
                if (user == null)
                {
                    user = new IdentityUser(adminUser)
                    {
                        Email = adminEmail,
                        EmailConfirmed = true // Одразу підтверджуємо
                    };
                    await userManager.CreateAsync(user, adminPassword);
                }

                // 4. Призначення ролі "Admin"
                if (!await userManager.IsInRoleAsync(user, adminRole))
                {
                    await userManager.AddToRoleAsync(user, adminRole);
                }
            }
        }
    }
}
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

Створення ViewModels для Account

```
using System.ComponentModel.DataAnnotations;

namespace SurveyPortal.Models.ViewModels
{
    public class LoginModel
    {
        [Required(ErrorMessage = "Введіть ім'я користувача")]
        [Display(Name = "Ім'я")]
        public string Name { get; set; } = string.Empty;

        [Required(ErrorMessage = "Введіть пароль")]
        [DataType(DataType.Password)]
        [Display(Name = "Пароль")]
        public string Password { get; set; } = string.Empty;

        public string ReturnUrl { get; set; } = "/";
    }
}
```

RegisterModel.cs Код:

```
using System.ComponentModel.DataAnnotations;

namespace SurveyPortal.Models.ViewModels
{
    public class RegisterModel
    {
        [Required(ErrorMessage = "Введіть ім'я користувача")]
        [Display(Name = "Ім'я")]
        public string UserName { get; set; } = string.Empty;

        [Required(ErrorMessage = "Введіть Email")]
        [EmailAddress]
        [Display(Name = "Email")]
        public string Email { get; set; } = string.Empty;

        [Required(ErrorMessage = "Введіть пароль")]
        [DataType(DataType.Password)]
        [Display(Name = "Пароль")]
        public string Password { get; set; } = string.Empty;

        [Required(ErrorMessage = "Підтвердіть пароль")]
        [DataType(DataType.Password)]
        [Display(Name = "Підтвердження")]
        [Compare("Password", ErrorMessage = "Паролі не співпадають")]
        public string ConfirmPassword { get; set; } = string.Empty;
    }
}
```

EditModel.cs Код:

```
using System.ComponentModel.DataAnnotations;

namespace SurveyPortal.Models.ViewModels
{
    public class EditModel
    {
        public string Id { get; set; } = string.Empty;

        [Required(ErrorMessage = "Введіть Email")]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [DataType(DataType.Password)]
    }
}
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		4

```

[Display(Name = "Новий пароль (необов'язково)")]
public string? Password { get; set; }

[DataType(DataType.Password)]
[Display(Name = "Підтвердження пароля")]
[Compare("Password", ErrorMessage = "Паролі не співпадають")]
public string? ConfirmPassword { get; set; }
}
}

Створення AccountController
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using SurveyPortal.Models.ViewModels;

namespace SurveyPortal.Controllers
{
    public class AccountController : Controller
    {
        private UserManager<IdentityUser> userManager;
        private SignInManager<IdentityUser> signInManager;
        private RoleManager<IdentityRole> roleManager; // Для роботи з ролями

        public AccountController(UserManager<IdentityUser> userMgr,
                                SignInManager<IdentityUser> signInMgr,
                                RoleManager<IdentityRole> roleMgr)
        {
            userManager = userMgr;
            signInManager = signInMgr;
            roleManager = roleMgr;
        }

        // --- LOGIN ---
        [AllowAnonymous] // Доступно всім
        public IActionResult Login(string returnUrl)
        {
            return View(new LoginModel
            {
                ReturnUrl = returnUrl ?? "/"
            });
        }

        [HttpPost]
        [AllowAnonymous]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Login(LoginModel loginModel)
        {
            if (ModelState.IsValid)
            {
                IdentityUser? user =
                    await userManager.FindByNameAsync(loginModel.Name);
                if (user != null)
                {
                    await signInManager.SignOutAsync();
                    var result = await signInManager.PasswordSignInAsync(user,
                        loginModel.Password, false, false);

                    if (result.Succeeded)
                    {
                        return Redirect(loginModel?.ReturnUrl ?? "/");
                    }
                }
                ModelState.AddModelError("", "Неправильне ім'я або пароль");
            }
        }
    }
}

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }
    return View(loginModel);
}

// --- REGISTER ---
[AllowAnonymous]
public ActionResult Register() => View(new RegisterModel());

[HttpPost]
[AllowAnonymous]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Register(RegisterModel model)
{
    if (ModelState.IsValid)
    {
        var user = new IdentityUser
        {
            UserName = model.UserName,
            Email = model.Email
        };

        var result = await userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            // АВТОМАТИЧНО ДОДАЄМО НОВОГО КОРИСТУВАЧА ДО РОЛІ "User"
            await userManager.AddToRoleAsync(user, "User");

            // Одразу логінімо
            await signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("Index", "Home");
        }
        else
        {
            foreach (var error in result.Errors)
            {
                ModelState.AddModelError("", error.Description);
            }
        }
    }
    return View(model);
}

// --- LOGOUT ---
[Authorize] // Доступно тільки залогіненим
public async Task<RedirectResult> Logout(string returnUrl = "/")
{
    await signInManager.SignOutAsync();
    return Redirect(returnUrl);
}

[Authorize]
public async Task<IActionResult> Edit()
{
    var user = await userManager.GetUserAsync(User); // Отримуємо поточного юзера
    if (user == null)
    {
        return RedirectToAction("Login");
    }

    var model = new EditModel
    {
        Id = user.Id,
        Email = user.Email
    };
}

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		6

```

        return View(model);
    }

    [HttpPost]
    [Authorize]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(EditModel model)
    {
        if (!ModelState.IsValid)
        {
            return View(model);
        }

        var user = await userManager.FindByIdAsync(model.Id);
        if (user == null)
        {
            ModelState.AddModelError("", "Користувача не знайдено");
            return View(model);
        }

        user.Email = model.Email;
        var emailResult = await userManager.UpdateAsync(user);

        if (!emailResult.Succeeded)
        {
            ModelState.AddModelError("", "Неможливо змінити Email.");
            return View(model);
        }

        if (!string.IsNullOrEmpty(model.Password))
        {
            var token = await userManager.GeneratePasswordResetTokenAsync(user);
            var passResult = await userManager.ResetPasswordAsync(user, token,
model.Password);

            if (!passResult.Succeeded)
            {
                ModelState.AddModelError("", "Неможливо змінити пароль.");
                foreach (var err in passResult.Errors)
                {
                    ModelState.AddModelError("", err.Description);
                }
                return View(model);
            }
        }

        TempData["message"] = "Профіль оновлено";
        return RedirectToAction("Index", "Home");
    }
}
}

```

Створення Login View

@model SurveyPortal.Models.ViewModels.LoginModel

<h2 class="mb-4">Вхід</h2>

<div asp-validation-summary="All" class="text-danger"></div>

<form asp-action="Login" asp-controller="Account" method="post">

<input type="hidden" asp-for="ReturnUrl" />

<div class="mb-3">

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        <label asp-for="Name" class="form-label"></label>
        <div><span asp-validation-for="Name" class="text-danger"></span></div>
        <input asp-for="Name" class="form-control" />
    </div>

    <div class="mb-3">
        <label asp-for="Password" class="form-label"></label>
        <div><span asp-validation-for="Password" class="text-danger"></span></div>
        <input asp-for="Password" class="form-control" />
    </div>

    <button class="btn btn-primary" type="submit">Увійти</button>
    <a asp-action="Register" asp-controller="Account" class="btn btn-
secondary">Реєстрація</a>
</form>

```

Створення Register View

@model SurveyPortal.Models.ViewModels.RegisterModel

```

<h2 class="mb-4">Реєстрація</h2>

<div asp-validation-summary="All" class="text-danger"></div>

<form asp-action="Register" asp-controller="Account" method="post">

    <div class="mb-3">
        <label asp-for="UserName" class="form-label"></label>
        <div><span asp-validation-for="UserName" class="text-danger"></span></div>
        <input asp-for="UserName" class="form-control" />
    </div>

    <div class="mb-3">
        <label asp-for="Email" class="form-label"></label>
        <div><span asp-validation-for="Email" class="text-danger"></span></div>
        <input asp-for="Email" class="form-control" />
    </div>

    <div class="mb-3">
        <label asp-for="Password" class="form-label"></label>
        <div><span asp-validation-for="Password" class="text-danger"></span></div>
        <input asp-for="Password" class="form-control" />
    </div>

    <div class="mb-3">
        <label asp-for="ConfirmPassword" class="form-label"></label>
        <div><span asp-validation-for="ConfirmPassword" class="text-danger"></span></div>
        <input asp-for="ConfirmPassword" class="form-control" />
    </div>

    <button class="btn btn-primary" type="submit">Зареєструватися</button>
    <a asp-action="Login" asp-controller="Account" class="btn btn-
secondary">Скасувати</a>
</form>

```

Створення Edit View (Особистий кабінет)

@model SurveyPortal.Models.ViewModels.EditModel

```
<h2 class="mb-4">Особистий кабінет</h2>

<div asp-validation-summary="All" class="text-danger"></div>

<form asp-action="Edit" asp-controller="Account" method="post">
  <input type="hidden" asp-for="Id" />

  <div class="mb-3">
    <label class="form-label">Ім'я користувача (не можна змінити)</label>
    <input value="@User.Identity.Name" class="form-control" disabled />
  </div>

  <div class="mb-3">
    <label asp-for="Email" class="form-label"></label>
    <div><span asp-validation-for="Email" class="text-danger"></span></div>
    <input asp-for="Email" class="form-control" />
  </div>

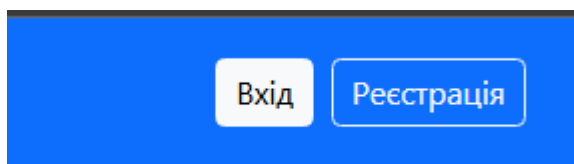
  <hr />
  <h5 class="mb-3">Зміна пароля</h5>

  <div class="mb-3">
    <label asp-for="Password" class="form-label"></label>
    <div><span asp-validation-for="Password" class="text-danger"></span></div>
    <input asp-for="Password" class="form-control" />
  </div>

  <div class="mb-3">
    <label asp-for="ConfirmPassword" class="form-label"></label>
    <div><span asp-validation-for="ConfirmPassword" class="text-danger"></span></div>
    <input asp-for="ConfirmPassword" class="form-control" />
  </div>

  <button class="btn btn-primary" type="submit">Зберегти зміни</button>
  <a asp-action="Index" asp-controller="Home" class="btn btn-secondary">Скасувати</a>
</form>
```

Скріншоти:



В хедері додано кнопки вхід\реєстрація

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		

Реєстрація

Ім'я

Email

Пароль

Підтвердження

Зареєструватися

Скасувати

Форма реєстрації

Особистий кабінет

Ім'я користувача (не можна змінити)

Oleg

Email

volunteer@test.com

Зміна пароля

Новий пароль (необов'язково)

Підтвердження пароля

Зберегти зміни

Скасувати

Особистий кабінет користувача

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
						10
Змн.	Арк.	№ докум.	Підпис	Дата		

Керування опитуваннями

Створити нове опитування

ID	Назва	Категорія	Рейтинг	Дії		
1	Опитування про спорт(тест виконання)	Спорт	5,00	Редагувати	Деталі	Видалити
2	Опитування про політику	Політика2	3,80	Редагувати	Деталі	Видалити
3	Опитування про фільми	Культура	4,50	Редагувати	Деталі	Видалити
7	іаін	вва	3,00	Редагувати	Деталі	Видалити

Вхід для адміністратора з додатковими можливостями

Завдання 2.

В рішенні створити новий WebAPI проект. Цей проект повинен реалізовувати всі функціональності, які наявні в MVC проекті у вигляді REST-ендпоїнтів. Для кращої організації і перевикористання коду слід перенести файли для роботи з БД (репозиторії, контексти, моделі) в окрему бібліотеку і додати посилання на неї в MVC та WebAPI проектах. Встановіть необхідні залежності для проекту бібліотеки. Після цього налаштуйте ConnectionString'и для створених БД для WebAPI проекту в файлі appsettings.json. Таким чином, після перенесення файлів для роботи з БД в окрему бібліотеку ми можемо використовувати всі необхідні сервіси в обох проектах.

Реалізуйте всі необхідні CRUD-операції в вашому WebAPI проекті, включаючи реєстрацію користувача. Перевірте працездатність ваших ендпоїнтів за допомогою Postman, Swagger або будь-яким іншим зручним інструментом.

1: Створення бібліотеки класів (Class Library)

1. Створити проект бібліотеки класів `dotnet new classlib -o SurveyPortal.Data` #
2. Додати новий проект до твого рішення (solution) `dotnet sln add SurveyPortal.Data/SurveyPortal.Data.csproj`

2: Додавання залежностей до бібліотеки

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
```

3: Переміщення файлів

Ім'я	Дата змінення	Тип	Розмір
 ApplIdentityDbContext.cs	13.11.2025 14:25	C# Source File	1 КБ
 EFSurveyRepository.cs	12.11.2025 10:51	C# Source File	2 КБ
 ISurveyRepository.cs	12.11.2025 10:51	C# Source File	1 КБ
 Question.cs	12.11.2025 11:31	C# Source File	1 КБ
 Survey.cs	12.11.2025 11:31	C# Source File	2 КБ
 SurveyDbContext.cs	12.11.2025 10:32	C# Source File	1 КБ

4: Оновлення Namespace у переміщених файлах

У кожному з 6 файлів, змінено namespace з SurveyPortal.Models на SurveyPortal.Data.Models

5: Оновлення проекту MVC (SurveyPortal)

```
Microsoft Windows [Version 10.0.26200.6899]
(c) Корпорація Майкрософт. Усі права захищені.

E:\University\4_course\Asp_net\lab2\SurveySln>dotnet add SurveyPortal/SurveyPortal.csproj reference SurveyPortal/SurveyPortal.Data/SurveyPortal.Data.csproj
Reference 'SurveyPortal.Data\SurveyPortal.Data.csproj' added to the project.

E:\University\4_course\Asp_net\lab2\SurveySln>dotnet remove SurveyPortal/SurveyPortal.csproj package Microsoft.EntityFrameworkCore.SqlServer
info : Removing PackageReference for package 'Microsoft.EntityFrameworkCore.SqlServer' from project 'SurveyPortal/SurveyPortal.csproj'.

E:\University\4_course\Asp_net\lab2\SurveySln>dotnet remove SurveyPortal/SurveyPortal.csproj package Microsoft.AspNetCore.Identity.EntityFrameworkCore
info : Removing PackageReference for package 'Microsoft.AspNetCore.Identity.EntityFrameworkCore' from project 'SurveyPortal/SurveyPortal.csproj'.

E:\University\4_course\Asp_net\lab2\SurveySln>d|
```

2: Створення Web API проекту

Створення проекту Web API

Виконання у терміналі:

```
E:\University\4_course\Asp_net\lab2\SurveySln>dotnet new webapi -o SurveyPortal.Api
The template "ASP.NET Core Web API" was created successfully.

Processing post-creation actions...
Restoring E:\University\4_course\Asp_net\lab2\SurveySln\SurveyPortal.Api\SurveyPortal.Api.csproj:
Restore succeeded.

E:\University\4_course\Asp_net\lab2\SurveySln>dotnet sln add SurveyPortal.Api/SurveyPortal.Api.csproj
Project 'SurveyPortal.Api\SurveyPortal.Api.csproj' added to the solution.

E:\University\4_course\Asp_net\lab2\SurveySln>dotnet add SurveyPortal.Api/SurveyPortal.Api.csproj reference SurveyPortal.Data/SurveyPortal.Data.csproj
Could not find project or directory 'SurveyPortal.Data\SurveyPortal.Data.csproj'.

E:\University\4_course\Asp_net\lab2\SurveySln>dotnet add SurveyPortal.Api/SurveyPortal.Api.csproj reference SurveyPortal/SurveyPortal.Data/SurveyPortal.Data.csproj
Reference '..\SurveyPortal\SurveyPortal.Data\SurveyPortal.Data.csproj' added to the project.
```

Налаштування appsettings.json для Web API

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "SurveyPortalConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=SurveyPortal;MultipleActiveResultSets=true",
    "IdentityConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=SurveyIdentity;MultipleActiveResultSets=true"
  }
}
```

9: Налаштування Program.cs для Web API

```
using Microsoft.EntityFrameworkCore;
using SurveyPortal.Data.Models;
using Microsoft.AspNetCore.Identity;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(); // Додаємо Swagger для тестування
builder.Services.AddCors(options =>
{
    options.AddDefaultPolicy(policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});

builder.Services.AddDbContext<SurveyDbContext>(opts => {
    opts.UseSqlServer(builder.Configuration["ConnectionStrings:SurveyPortalConnection"]);
});
builder.Services.AddDbContext<AppIdentityDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration["ConnectionStrings:IdentityConnection"])
);
builder.Services.AddScoped<ISurveyRepository, EFSurveyRepository>();

builder.Services.AddAuthentication();
builder.Services.AddAuthorization();

builder.Services.AddIdentityCore<IdentityUser>(opts =>
{
    opts.Password.RequiredLength = 8;
    opts.Password.RequireDigit = true;
    opts.Password.RequireUppercase = true;
    opts.User.RequireUniqueEmail = true;
})
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<AppIdentityDbContext>()
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		13

```

        .AddApiEndpoints();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseCors();

app.UseAuthentication();
app.UseAuthorization();

Створює ендпоїнти Identity (напр. /register, /login)
app.MapIdentityApi<IdentityUser>();

// 8. Підключає наші контролери
app.MapControllers();

app.Run();

```

10: Створення SurveysApiController

```

using Microsoft.AspNetCore.Mvc;
using SurveyPortal.Data.Models;
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Authorization;

namespace SurveyPortal.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class SurveysApiController : ControllerBase
    {
        private readonly ISurveyRepository _repository;

        public SurveysApiController(ISurveyRepository repository)
        {
            _repository = repository;
        }

        // READ (List)
        // GET: /api/SurveysApi
        [HttpGet]
        [AllowAnonymous] // Дозволити всім бачити список
        public async Task<ActionResult<IEnumerable<Survey>>> GetSurveys()
        {
            return await _repository.Surveys.ToListAsync();
        }

        // READ (Details)
        // GET: /api/SurveysApi/5
        [HttpGet("{id}")]
        [AllowAnonymous]
        public async Task<ActionResult<Survey>> GetSurvey(long id)
        {
            var survey = await _repository.Surveys.FirstOrDefaultAsync(s => s.SurveyID ==
id);

            if (survey == null)

```

```

        {
            return NotFound();
        }

        return survey;
    }

    [HttpPost]
    [Authorize(Roles = "Admin")]
    public async Task<ActionResult<Survey>> PostSurvey(Survey survey)
    {
        _repository.SaveSurvey(survey);

        return CreatedAtAction(nameof(GetSurvey), new { id = survey.SurveyID },
survey);
    }
    [HttpPut("{id}")]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> PutSurvey(long id, Survey survey)
    {
        if (id != survey.SurveyID)
        {
            return BadRequest();
        }

        _repository.SaveSurvey(survey);
        return NoContent();
    }
    [HttpDelete("{id}")]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> DeleteSurvey(long id)
    {
        var deletedSurvey = _repository.DeleteSurvey(id);
        if (deletedSurvey == null)
        {
            return NotFound();
        }
        return NoContent(); // Усіх 204
    }
}

```

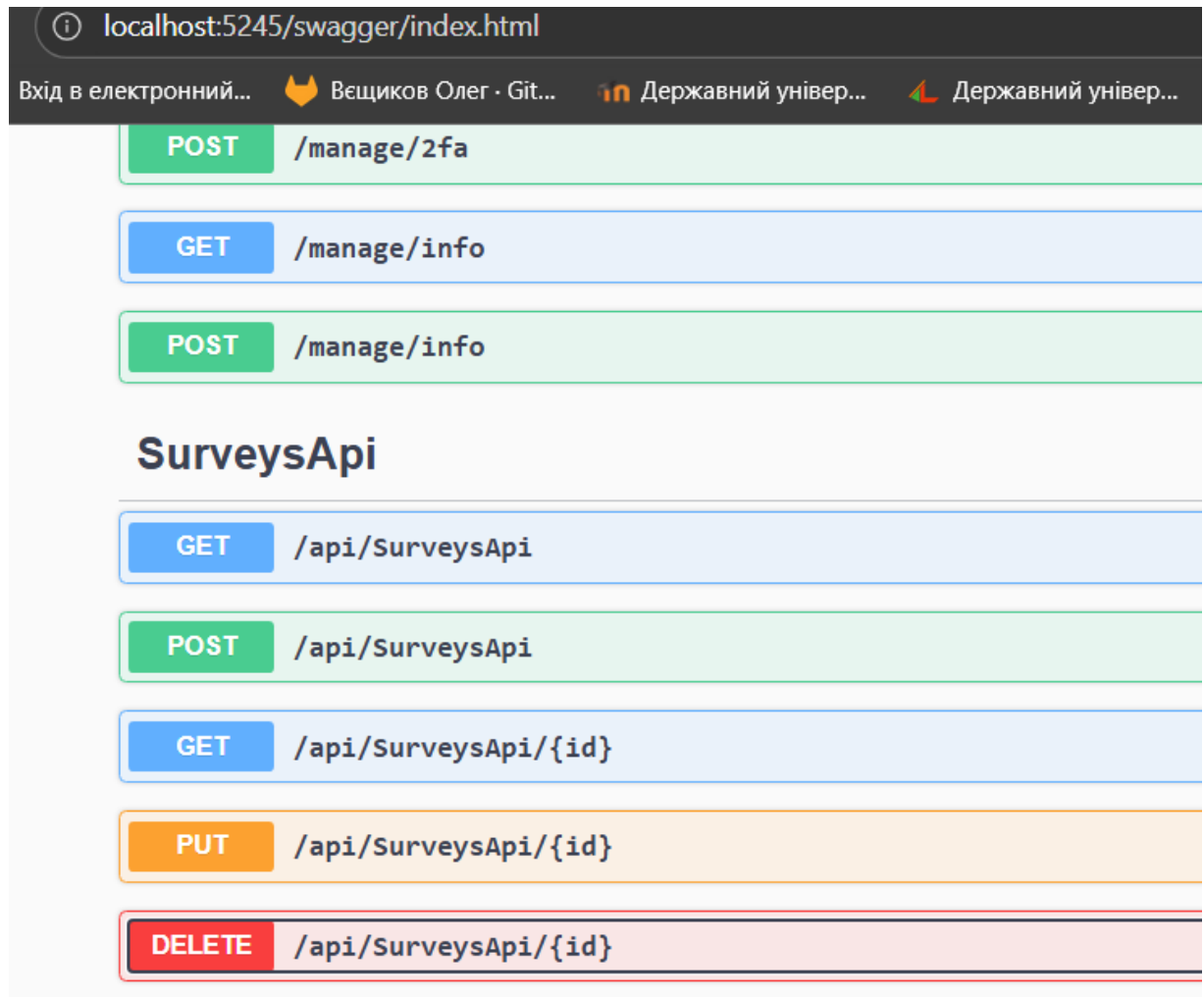
Створено контролер `SurveysApiController`, який реалізує CRUD-операції, використовуючи HTTP-методи:

1. GET `/api/SurveysApi` — Отримання списку всіх опитувань (доступно всім).
2. GET `/api/SurveysApi/{id}` — Отримання детальної інформації про опитування (доступно всім).
3. POST `/api/SurveysApi` — Створення нового опитування (тільки для ролі Admin).
4. PUT `/api/SurveysApi/{id}` — Оновлення опитування (тільки для ролі Admin).
5. DELETE `/api/SurveysApi/{id}` — Видалення опитування (тільки для ролі Admin).

Для захисту методів використано атрибут [Authorize(Roles = "Admin")].

Перевірка працездатності (Swagger UI)

Проект успішно запущено. Для тестування API використовувався вбудований інтерфейс Swagger UI.



GET /api/SurveysApi

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5245/api/SurveysApi' \
  -H 'accept: text/plain'
```

Request URL

http://localhost:5245/api/SurveysApi

Server response

Code

Details

200

Response body

```
{
  "questions": [],
  "surveyID": 2,
  "title": "Опитування про політику",
  "description": "Що ви думаєте про вибори та виборців?",
  "creator": "Admin",
  "category": "Політика2",
  "averageRating": 3.8,
  "questions": []
}
```

Тестування роботи за вибором id:

GET /api/SurveysApi/{id}

Parameters

Name	Description
id * required	
integer(\$int64)	2
(path)	

Execute Clear

Responses

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5245/api/SurveysApi/2' \
  -H 'accept: text/plain'
```

Request URL

http://localhost:5245/api/SurveysApi/2

Server response

Code

Details

200

Response body

```
{
  "surveyID": 2,
  "title": "Опитування про політику",
  "description": "Що ви думаєте про вибори та виборців?",
  "creator": "Admin",
  "category": "Політика2",
  "averageRating": 3.8,
  "questions": []
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 18 Nov 2025 09:38:16 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200		No links

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		17

Вхід в ак адміна в залежності від версії Swagger

POST /login

Parameters

Name	Description
useCookies boolean (query)	--
useSessionCookies boolean (query)	--

Request body

application/json

```
{  "email": "admin@survey.com",  "password": "AdminPassword123"}
```

Результати перевірки:

GET-запити успішно повертають дані у форматі JSON.

Захищені методи: При спробі виконати DELETE без токена отримано статус 401

1. Встановлення необхідних залежностей

Для роботи з токенами у проект SurveyPortal.Api було додано пакет Microsoft.AspNetCore.Authentication.JwtBearer.

Команда встановлення:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer --version 8.0.0
```

2. Налаштування автентифікації в Program.cs

У файлі конфігурації Program.cs було налаштовано сервіси автентифікації. Оскільки ми використовуємо .NET 8 Identity API, було застосовано сучасний підхід з використанням AddApiEndpoints та AddBearerToken.

Ключові зміни:

Вказано схему автентифікації за замовчуванням як IdentityConstants.BearerScheme.

Додано обробник AddBearerToken, який дозволяє API валідувати токени, видані вбудованим ендпоінтом /login.

Лістинг конфігурації:

```
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = IdentityConstants.BearerScheme;
    options.DefaultChallengeScheme = IdentityConstants.BearerScheme;
})
.AddBearerToken(IdentityConstants.BearerScheme);

builder.Services.AddAuthorization();

builder.Services.AddIdentityCore<IdentityUser>()
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<AppIdentityDbContext>()
    .AddApiEndpoints(); // Додає підтримку токенів
```

3. Налаштування Swagger для роботи з JWT

Щоб мати змогу тестувати захищені методи прямо через Swagger UI, було додано конфігурацію AddSwaggerGen. Вона додає кнопку "Authorize" та дозволяє передавати заголовок Authorization: Bearer <token>.

Лістинг налаштування Swagger:

```
builder.Services.AddSwaggerGen(c =>
{
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Description = "Введіть токен у форматі: Bearer {ваш_токен}",
```

```

        Name = "Authorization",

        In = ParameterLocation.Header,

        Type = SecuritySchemeType.ApiKey,

        Scheme = "Bearer"

    });

});

```

4. Захист контролерів

До контролера `SurveysApiController` було додано атрибут `[Authorize]`, щоб обмежити доступ до методів модифікації даних (`Create`, `Update`, `Delete`) лише для автентифікованих користувачів з роллю `"Admin"`.

```

[HttpPost]

[Authorize(Roles = "Admin")] // Доступ тільки з валідним токеном адміна

public ActionResult<Survey> PostSurvey(Survey survey)

```

5. Перевірка працездатності

Тестування проводилося за допомогою Swagger UI.

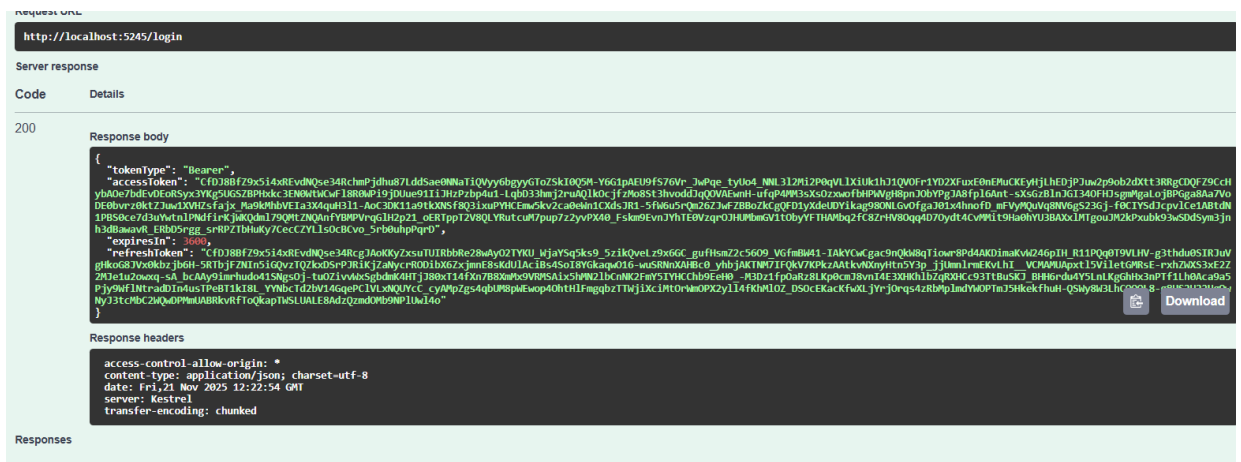
Етап А: Автентифікація (Отримання токена) Було відправлено POST-запит на ендпоінт `/login` з даними адміністратора.

Запит: `{"email": "admin@survey.com", "password": "Admin_P@ssw0rd123!"}`

Результат: Сервер повернув статус `200 OK` та JSON-об'єкт з полем `accessToken`.

Приклад отриманої відповіді (зі Swagger):

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.121.8.000 – Лр.5	Арк.
						20
Змн.	Арк.	№ докум.	Підпис	Дата		



Етап Б: Виконання захищеного запиту

Available authorizations

×

Bearer (apiKey)

Введіть токен у форматі: Bearer {ваш_токен}

Name: Authorization

In: header

Value:

:ZYLIsOcBCvo_5rb0uhpPqrD

AuthorizeClose

Отриманий токен було скопійовано та вставлено у вікно авторизації Swagger (Bearer <token>).

POST /api/SurveysApi

Parameters

No parameters

Request body

application/json

```
{  "surveyID": 5,  "title": "Опитування: Вподобання у їжі",  "description": "Ми хочемо дізнатися, який кушні ви надасте перевагу для нового корпоративу.",  "creator": "Admin",  "category": "Кушні",  "averageRating": 5,  "questions": [    {      "questionID": 0,      "text": "Яка ваша улюблена кушні?",      "questionType": "Text"    },    {      "questionID": 0,      "text": "Чи є у вас алергія на певні продукти?",      "questionType": "YesNo"    },    {      "questionID": 0    }  ]}
```

ExecuteClear

Responses

201
Undocumented

Response body

```

{
  "surveyID": 5,
  "title": "Опитування: Вподобання у їжі",
  "description": "Ми хочемо дізнатися, якій кухні ви надаєте перевагу для нашого корпоративу.",
  "creator": "Admin",
  "category": "Кухня",
  "averageRating": 5,
  "questions": [
    {
      "questionID": 0,
      "text": "Яка ваша улюблена кухня?",
      "questionType": "Text",
      "surveyID": 0,
      "survey": null
    },
    {
      "questionID": 0,
      "text": "Чи є у вас алергія на певні продукти?",
      "questionType": "YesNo",
      "surveyID": 0,
      "survey": null
    },
    {
      "questionID": 0,
      "text": "Оцініть італійську кухню від 1 до 5",
      "questionType": "Rating",
      "surveyID": 0,
      "survey": null
    }
  ]
}

```

Response headers

```

access-control-allow-origin: *
content-type: application/json; charset=utf-8
date: Sat, 22 Nov 2025 13:06:50 GMT
location: http://localhost:5245/api/SurveysApi/5

```

Виконано запит POST /api/SurveysApi для створення нового опитування.

Результат: Запит успішно пройшов перевірку токена, і сервер створив ресурс (код 201 Created). Спроба виконати цей же запит без токена завершилася помилкою 401 Unauthorized.

Висновок:

У ході лабораторної роботи було успішно реалізовано механізм JWT-автентифікації. Web API тепер підтримує безпечний обмін даними: клієнти отримують токен доступу після успішного входу і використовують його для підтвердження своїх прав при зверненні до захищених ресурсів. Swagger UI налаштовано для зручного тестування цього процесу.